

```
root@DESKTOP-GEP6I4T: ~ X + v
root@DESKTOP-GEP6I4T:~# nano token.c
root@DESKTOP-GEP6I4T:~# gcc token.c -o token
root@DESKTOP-GEP6I4T:~# ./token
Enter command: ls -l home/user

Tokens:
Token 1: ls
Token 2: -l
Token 3: home/user

Token stream is valid.
root@DESKTOP-GEP6I4T:~# cat token.c

#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 20

int main() {
    char input[200];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Split using whitespace as delimiter
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \t");
    }

    // Display tokens
    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
```

```
root@DESKTOP-GE6I4T: ~  
root@DESKTOP-GE6I4T:~# nano parser.c  
root@DESKTOP-GE6I4T:~# gcc parser.c -o parser  
root@DESKTOP-GE6I4T:~# ./parser  
Enter command: ls -l file.txt  
  
Parse Tree:  
COMMAND  
|-- ls  
|-- OPTION: -l  
|-- ARGUMENT: file.txt  
  
Syntax: VALID  
root@DESKTOP-GE6I4T:~# cat parser.c  
#include <stdio.h>  
#include <string.h>  
  
#define MAX_TOKENS 20  
  
typedef struct Node {  
    char value[50];  
    struct Node *next;  
} Node;  
  
void parse(char *tokens[], int count) {  
  
    if (count == 0) {  
        printf("Empty command.\n");  
        return;  
    }  
  
    printf("\nParse Tree:\n");  
    printf("COMMAND\n");  
    printf("-- %s\n", tokens[0]);  
  
    for (int i = 1; i < count; i++) {  
  
        if (tokens[i][0] == '-') {  
            printf("-- OPTION: %s\n", tokens[i]);  
        }  
        else {  
            printf("-- ARGUMENT: %s\n", tokens[i]);  
        }  
    }  
}
```